



Programação Avançada em Java

Trabalho Prático 5

Curso: Acertar o Rumo

Introdução

Objetivo do Trabalho

Este Trabalho Prático 5 constrói sobre o Projeto 4, introduzindo novas funcionalidades típicas de aplicações reais, com ênfase em:

- Robustez e segurança
- Comunicação em tempo real
- Boas práticas de REST API e WebSockets
- Experiência de utilizador fluída e responsiva

Informações Gerais

Data de Entrega: 5 de maio de 2026

Entrega Individual: Sim

Penalizações:

- Diferenças entre código entregue e demonstrado: -25%
- Falta de preparação na defesa: -10%
- Componentes não demonstrados ou incompletos: -35%

Ao concluir este projeto, os alunos deverão implementar funcionalidades como:

- Confirmação de contas por e-mail e recuperação de password
- Sistema de mensagens e notificações em tempo real
- Sessões com timeout configurável
- Internacionalização (i18n) e responsividade
- Dashboard de administrador com estatísticas e gráficos

- Logs de atividades
- Comunicação eficiente via REST e WebSockets

Requisitos Gerais

O projeto deve respeitar todos os requisitos do Projeto 4:

- Frontend em ReactJS comunicando com Backend Jakarta EE (JAX-RS, EJBs)
- Persistência de dados em PostgreSQL com JPA (Hibernate)
- Entidades base: Utilizadores, Clientes e Leads
- Controlo de acesso (diferentes tipos de utilizador)
- Boas práticas de REST API e comunicação
- Filtragem e ordenação realizadas no backend

Novos Requisitos

Módulo	Funcionalidade	Tipo
Confirmação de Contas	Confirmação via e-mail, link único e expiração	Segurança
Recuperação de Password	Reset via link único, token expira após uso	Segurança
Perfil do Utilizador	Página única por URL, estatísticas de leads	Interface
Listagem de Utilizadores	Página filtrável, redirecionamento para perfil	Interface
Mensagens e Notificações	Mensagens 1-para-1, contador de notificações, histórico	Tempo real
Internacionalização	PT e EN, preferências por utilizador	Interface
Dashboard Admin	Estatísticas, gráficos, leads por utilizador	Interface/Admin
Atualizações em Tempo Real	WebSockets para chat, notificações e dashboard	Tempo real
Session Timeout	Logout automático após inatividade	Segurança
Logs	Registo de atividades (CRUD, login, etc.)	Auditoria
Responsividade	Mobile, Tablet, Desktop	Interface
Comunicação Eficiente	REST e WS enviando apenas dados necessários	Backend

Funcionalidades Detalhadas

1. Sistema de Confirmação de Contas

Confirmação de Contas

- Administrador cria contas fornecendo e-mail
- Conta permanece em estado “Aguardando Confirmação” até o utilizador clicar no link recebido
- Antes da confirmação, o utilizador não pode interagir com clientes ou leads
- API deve retornar erro 403 caso haja tentativa de operação antes da confirmação
- Tokens devem expirar após X horas e ser inválidos após uso

2. Recuperação de Password

Recuperação de Password

- Página “Esqueceu a password?” solicita e-mail do utilizador
- Envio de link único para redefinição
- Token expira após uso ou tempo definido
- Após redefinição, token é invalidado

3. Perfil do Utilizador

Perfil do Utilizador

- URL único por utilizador (e.g., /users/username)
- Informações: nome, username, e-mail, fotografia
- Estatísticas de leads associadas
- Botão para editar configurações pessoais
- Apenas o próprio utilizador ou Administrador pode editar

4. Listagem de Utilizadores

Listagem de Utilizadores

- Mostrar username, fotografia e e-mail
- Filtro por username ou e-mail (backend)
- Clique redireciona para perfil do utilizador

5. Chat e Notificações

Mensagens e Notificações

- Mensagens 1-para-1 com histórico
- Mensagens inicialmente não lidas; marcar como lidas ao abrir
- WebSockets para receção em tempo real; REST (POST) para envio inicial
- Notificações com contador sempre visível
- Notificações marcadas como lidas ao abrir lista
- Canal isolado por utilizador para preservar privacidade

6. Internacionalização e Localização

Internacionalização

- Mínimo de 2 idiomas: PT e EN
- Preferência de idioma configurável no perfil do utilizador
- Tradução de UI via ficheiros JSON
- Conteúdo dinâmico (leads, nomes) não precisa de tradução

7. Dashboard do Administrador

Dashboard

- Estatísticas: total utilizadores, clientes, leads, contas confirmadas
- Leads por estado e por utilizador
- Dashboard estático (sem WebSockets)
- Gráficos de evolução temporal de utilizadores e leads

8. Logs e Auditoria

Logs

- Registo de atividades relevantes (CRUD, login, etc.)
- Cada entrada inclui data, hora, utilizador/token

9. Responsividade e Interface

Responsividade

- Layout adaptativo para Mobile, Tablet e Desktop
- Abordagem mobile-first recomendada

10. Comunicação Eficiente

Comunicação Eficiente

- REST e WebSocket devem enviar apenas dados necessários
- Filtragem, ordenação e paginação sempre no backend

Testes de Integração e Robustez (Extra)

Extra: Selenium e Testes Automatizados

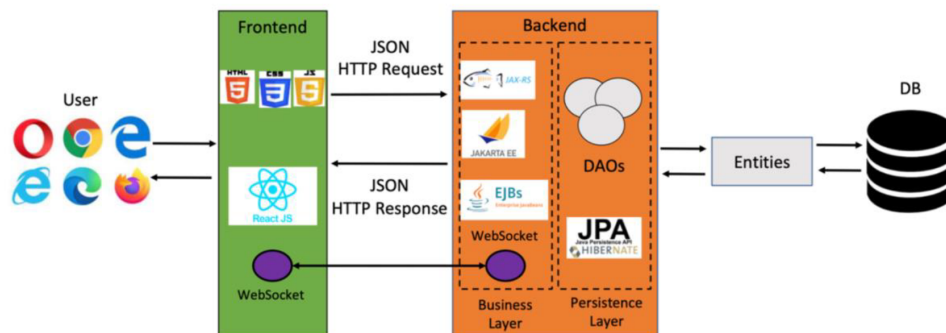
Objetivo: Avaliar a robustez e integração da aplicação em ambiente real, garantindo que os principais fluxos funcionam corretamente em diferentes browsers e cenários.

- Implementar até 30 testes automatizados usando **Selenium WebDriver**.
- Testes sugeridos:
 1. Confirmação de contas via link de e-mail.
 2. Recuperação de password com token expirado e válido.
 3. Login e logout automático após session timeout.
 4. Criação, edição e eliminação de leads.
 5. Envio e receção de mensagens 1-para-1.
 6. Atualização do dashboard do admin.
 7. Internacionalização: troca de idioma na UI e persistência.
 8. Responsividade: verificação de layouts Mobile, Tablet e Desktop.
 9. Visualização e navegação em perfil de utilizador.
 10. Contagem de notificações corretamente atualizada.
- Cada teste deve verificar o resultado esperado com assertivas Selenium.
- Avaliação: **valor máximo extra 1 ponto**, proporcional ao número de testes implementados e verificados.
- Entrega: scripts Selenium no repositório, com instruções para execução.

Exemplo de Teste Selenium em Java:

```
WebDriver driver = new ChromeDriver();
driver.get("http://localhost:8080/login");
driver.findElement(By.id("username")).sendKeys("testuser");
driver.findElement(By.id("password")).sendKeys("password123");
driver.findElement(By.id("loginBtn")).click();
assertTrue(driver.findElement(By.id("dashboard")).isDisplayed());
driver.quit();
```

Arquitetura e Tecnologias



REST API e Repositórios

- JSON como formato de comunicação
- Endpoints RESTful
- Backend e frontend em repositórios Git independentes

Bibliotecas Externas

- WebSockets, gráficos, ou outras com aprovação docente

Entrega do Projeto e Avaliação

Entregas e Defesa

Entrega: Guardar projeto em repositório Git. Apenas o último commit é considerado.

Defesa: 30 minutos, inscrição no InforEstudante, código funcional. Penalizações:

- Diferenças entre código entregue e demonstrado: -25%
- Falta de preparação: -10%
- Componentes não demonstrados ou incompletos: -35%

Q&A (Perguntas Frequentes)

- **Como gerir consistência de dados entre REST e WebSockets?**
Garantir que todas as atualizações via WebSocket refletem no estado persistido, evitando duplicação ou perda de dados.
- **Como tratar sessões expiradas durante comunicação em tempo real?**
O servidor rejeita pedidos WebSocket inválidos e notifica o cliente para login.
- **Como estruturar tokens de recuperação e confirmação de conta?**
Tokens únicos, temporários, invalidados após uso.

- **Como implementar internacionalização sem impactar performance?**
Carregar apenas ficheiros JSON correspondentes ao idioma do utilizador; traduzir apenas UI.
- **Como garantir que as estatísticas do dashboard são precisas em tempo real?**
Usar agregações SQL eficientes e atualizar apenas valores alterados via WebSocket.
- **Como lidar com mensagens simultâneas de múltiplos utilizadores?**
Canal isolado por utilizador, garantindo entrega correta e preservação de privacidade.
- **Como testar funcionalidades incompletas ou casos extremos?**
Documentar limitações; funcionalidades não demonstradas recebem penalização.
- **Como garantir segurança e performance em grandes volumes de dados?**
Paginação, filtros no backend, envio seletivo via WebSocket, evitar payloads grandes.